

8. Polymorphism

Polymorphism (Poly = Many, Morphism = Forms) is the phenomenon of using the same operator or method to perform two or more operations.

Example with '+' operator:

- SVD (addition): $20 + 10 = 30$
- MVD (concatenation): `'Hello' + 'World' = 'HelloWorld'`

Example with '*' operator:

- SVD (multiplication): $5 * 5 = 25$
- MVD (duplication): `[10, 20] * 2 = [10, 20, 10, 20]`

Polymorphism can be done in 2 ways:

- Method Overloading
- Operator Overloading

Method Overloading

Using the same method name to perform two or more different operations. In Python, if method overloading is done, it acts as method overriding (last definition wins).

Method Overriding

When the same function name is used for two or more operations, the first function's address gets overridden by the address of the next function.

Note: To access the previous overridden method, use the concept of Monkey Patching.

Monkey Patching

The process of storing the previous method's address into a variable and accessing that method using that variable.

```
var = add # Store old address
# Then redefine add
# Use var() to call old version
```

Operator Overloading

Making operators work on objects of user-defined class or user-defined datatypes by invoking the respective magic method (dunder).

Magic Methods: Methods defined inside a class with double underscores on both sides (`__method__`).

Operator / Operation	Operator Symbol	Internal Syntax (Magic Method)
----------------------	-----------------	--------------------------------

Addition (+)	obj1 + obj2	(obj1).__add__(obj2)
Subtraction (-)	obj1 - obj2	(obj1).__sub__(obj2)
Multiplication (*)	obj1 * obj2	(obj1).__mul__(obj2)
True Division (/)	obj1 / obj2	(obj1).__truediv__(obj2)
Floor Division (//)	obj1 // obj2	(obj1).__floordiv__(obj2)
Modulus (%)	obj1 % obj2	(obj1).__mod__(obj2)
Exponential (**)	obj1 ** obj2	(obj1).__pow__(obj2)
Equal to (==)	obj1 == obj2	(obj1).__eq__(obj2)
Not equal to (!=)	obj1 != obj2	(obj1).__ne__(obj2)
Greater than (>)	obj1 > obj2	(obj1).__gt__(obj2)
Less than (<)	obj1 < obj2	(obj1).__lt__(obj2)

Example – Operator Overloading

```
class Temp:
    def __init__(self, arg):
        self.arg = arg

    def __add__(self, var):
        return self.arg + var.arg

ob1 = Temp(10)
ob2 = Temp(20)
print(ob1 + ob2)    # 30
```

9. Encapsulation

Wrapping up the data and code into a single unit to provide security by using access specifiers (like a capsule containing different medicines).

Types of Access Specifiers

- Public Access Specifier
- Protected Access Specifier
- Private Access Specifier

Public Access Specifier

- Public members can be accessed outside the class.
- Normal members created in a normal class act as public members.

Protected Access Specifier

- Members that should provide security to data inside the class.

- Prefix: single underscore (_) – called 'soft barrier'.
- Note: In Python, protected members provide no actual security.

Private Access Specifier

- Members that provide security to data inside the class.
- Cannot be accessed or modified outside the class directly.
- To create a private member, use double underscore (__) before the variable or method name.

Accessing Private Members Outside the Class

Three ways to access or modify private members outside the class:

1. Direct Syntax:

```
obj/Cname.__Cname__var/mname(arg)
obj/Cname.__Cname__var = new_value
```

2. Using get and set methods:

```
class Temp:
    __var = 10

    def get(self):
        print(self.__var)

    def set(self, new_val):
        self.__var = new_val
```

```
ob = Temp()
ob.get()    # 10
ob.set(20)
ob.get()    # 20
```

10. Abstraction

Hiding the implementation from the user and showing only the functionality. Example: ATM machine — the user sees balance/withdraw/deposit options but not the internal code.

3 Important Terms

- Abstract Method
- Abstract Class
- Concrete Class

Abstract Method: A method that contains a function declaration but not a function definition (body contains only 'pass').

```
def fname(args):
    pass
```

Abstract Class: A class that consists of at least one abstract method. Objects cannot be created for an abstract class.

Concrete Class: A class that does not consist of any abstract method. Objects can be created for a concrete class.

General Syntax

```
from abc import ABC, abstractmethod

class Cname(ABC):
    @abstractmethod
    def Mname(args):
        pass
# This is an Abstract Class

class Cname2(Cname1):
    def mname(args):
        Statement Block
# This is a Concrete Class
```

Example

```
from abc import ABC, abstractmethod

class ATM(ABC):
    @abstractmethod
    def deposit(self): pass

    @abstractmethod
    def withdraw(self): pass

    @abstractmethod
    def ch_bal(self): pass

class MyATM(ATM):
    def deposit(self):
        print('Depositing Money')

    def withdraw(self):
        print('Withdrawing Money')

    def ch_bal(self):
        print('Checking balance')

atm = MyATM()
atm.deposit()
atm.withdraw()
atm.ch_bal()
```